

AIML QA Guide

- [AI/ML Intern Assessment — Questions & Answers](#)
 - [SECTION A — Multiple Choice \(15 × 1 = 15 marks\)](#)
 - [Q1.](#)
 - [Q2.](#)
 - [Q3.](#)
 - [Q4.](#)
 - [Q5.](#)
 - [Q6.](#)
 - [Q7.](#)
 - [Q8.](#)
 - [Q9.](#)
 - [Q10.](#)
 - [Q11.](#)
 - [Q12.](#)
 - [Q13.](#)
 - [Q14.](#)
 - [Q15.](#)
 - [SECTION B — Theory & Coding \(5 × 15 = 75 marks\)](#)
 - [Q16. End-to-End Machine Learning Pipeline \(15 marks\)](#)
 - [Q17. Deep Learning with PyTorch/TensorFlow \(15 marks\)](#)
 - [Q18. Generative AI — Building a RAG System \(15 marks\)](#)
 - [Q19. Model Deployment & AI/ML Engineering \(15 marks\)](#)
 - [Q20. Automation & Agentic AI with n8n \(15 marks\)](#)
 - [Last-minute priority if time is short](#)

AI/ML Intern Assessment — Questions & Answers

Full questions paired with answers and explanations, so you can study each one as a complete unit.

SECTION A — Multiple Choice (15 × 1 = 15 marks)

Q1.

You have a DataFrame `df` with columns `city` and `sales`. Which expression returns the total sales per city, sorted from highest to lowest?

- A. `df.groupby('city')['sales'].sum().sort_values(ascending=False)`
- B. `df.groupby('sales')['city'].sum().sort_values()`
- C. `df['sales'].sum().groupby('city').sort_values(ascending=False)`
- D. `df.sort_values('sales').groupby('city').mean()`

Answer: A Group by the category column, aggregate the numeric column, then sort. B/C/D have the group/aggregate order backwards or apply `sum()` before grouping, which doesn't work as intended.

Q2.

Consider the SQL query:

```
SELECT department, COUNT(*) AS emp_count
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 5;
```

Which statement is correct?

- A. HAVING filters individual rows before grouping, WHERE filters groups after grouping
- B. WHERE filters rows before grouping, HAVING filters the grouped result
- C. The query is invalid because WHERE and HAVING cannot appear together
- D. HAVING COUNT() > 5 can be replaced by WHERE COUNT() > 5 with identical results

Answer: B WHERE filters individual rows **before** grouping happens; HAVING filters **after** grouping, on aggregated values (like COUNT(*)). You can't write WHERE COUNT(*) > 5 because COUNT(*) doesn't exist yet at the row-filtering stage — that's exactly why HAVING exists.

Q3.

A client calls your REST API and receives HTTP status 422 with a JSON body containing a detail field. What is the most likely cause?

- A. The server crashed while processing the request
- B. The endpoint URL does not exist
- C. The request body was well-formed JSON but failed validation (wrong type / missing field)
- D. The client is not authenticated

Answer: C HTTP 422 = "Unprocessable Entity" — the JSON was syntactically valid but failed validation (wrong type, missing required field). 400 = malformed request, 404 = not found, 401/403 = auth issues.

Q4.

Which of the following is an example of **data leakage**?

- A. Fitting StandardScaler on the full dataset, then splitting into train and test
- B. Fitting StandardScaler on the training set only and using it to transform the test set
- C. Using stratify=y in train_test_split
- D. Using a fixed random_state in train_test_split

Answer: A Fitting StandardScaler on the **full dataset** before splitting lets test-set statistics (mean/std) leak into training — the model indirectly "sees" test data. Always fit preprocessing on train only, then transform test with those same parameters.

Q5.

You are cross-validating a binary classifier on a dataset where only 3% of samples belong to the positive class. Which strategy is most appropriate?

- A. KFold(shuffle=False)
- B. StratifiedKFold
- C. A single 80/20 random split

- D. `LeaveOneOut` on the majority class only

Answer: B `StratifiedKFold` preserves the class ratio (3% positive) in every fold. Plain `KFold` or a single random split risks folds with zero or very few positive examples, making evaluation unstable/meaningless.

Q6.

A fraud detection model reports 99.2% accuracy on a dataset where 0.8% of transactions are fraudulent, but it catches almost no fraud. Which metric pair best exposes this problem?

- A. Accuracy and R^2
- B. Precision and Recall (or PR-AUC)
- C. Mean Squared Error and MAE
- D. Silhouette score and inertia

Answer: B With 99.2% of the data being “not fraud,” a model that predicts “not fraud” every time scores ~99% accuracy while catching zero fraud. Precision/Recall exposes that it’s not actually detecting the minority class.

Q7.

Training accuracy is 99% and validation accuracy is 68%, with the gap widening each epoch. Which action is **least** likely to help?

- A. Adding dropout or L2 regularisation
- B. Collecting more training data / applying augmentation
- C. Increasing model depth and width
- D. Early stopping based on validation loss

Answer: C Increasing model depth/width **least** helps — that’s classic overfitting (huge train/val gap), and more capacity makes overfitting worse. Dropout, regularization, more data, and early stopping all directly fight overfitting.

Q8.

Which statement about K-Means clustering is **true**?

- A. It automatically determines the optimal number of clusters
- B. It assumes roughly spherical, similarly sized clusters and is sensitive to feature scaling
- C. It can natively label noise points as outliers
- D. It always converges to the global optimum regardless of initialisation

Answer: B K-Means assumes roughly spherical, similarly-sized clusters and **is** sensitive to feature scaling (unscaled features with large ranges dominate distance calculations). It does *not* auto-pick k, doesn’t natively detect outliers/noise (that’s DBSCAN), and can converge to different local optima depending on initialization.

Q9.

An input tensor of shape `(1, 3, 32, 32)` is passed through `nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1)` followed by `nn.MaxPool2d(2)`. What is the output shape?

- A. `(1, 16, 32, 32)`
- B. `(1, 16, 16, 16)`

- C. (1, 3, 16, 16)
- D. (1, 16, 15, 15)

Answer: B `padding=1, kernel=3, stride=1` keeps spatial size the same (32×32) after conv, but changes channels $3 \rightarrow 16$. Then `MaxPool2d(2)` halves spatial dims: $32 \rightarrow 16$. So: (batch=1, channels=16, H=16, W=16).

Q10.

In PyTorch, `nn.CrossEntropyLoss()` is used for multi-class classification. Which is correct?

- A. The model's final layer must apply softmax, because the loss expects probabilities
- B. The model should output raw logits; softmax (log-softmax) is applied inside the loss
- C. Targets must be one-hot encoded float tensors
- D. It is equivalent to `nn.MSELoss()` on one-hot targets

Answer: B `CrossEntropyLoss` expects **raw logits** — it internally applies log-softmax + NLL loss. If you also apply softmax in your model's final layer, you're double-applying it, which breaks training. Targets are integer class indices, not one-hot floats.

Q11.

Why do Transformer models need positional encodings?

- A. To reduce the number of parameters in the attention layers
- B. Because self-attention is permutation-invariant and otherwise has no notion of token order
- C. To normalise gradients during backpropagation
- D. To limit the context window to a fixed length

Answer: B Self-attention has no inherent sense of sequence order (it treats tokens as an unordered set mathematically) — positional encodings inject that order information explicitly.

Q12.

Two text chunks are converted into embeddings. Which is the standard way to measure their semantic similarity in a vector database?

- A. Levenshtein (edit) distance between the raw strings
- B. Cosine similarity between the embedding vectors
- C. Exact keyword match count
- D. Difference in vector dimensionality

Answer: B Cosine similarity between embedding vectors is the standard measure of semantic closeness in vector databases (measures angle/direction, not raw string similarity).

Q13.

Which scenario is the strongest case for **RAG** rather than fine-tuning?

- A. You need the model to consistently reply in a fixed JSON style
- B. You need answers grounded in a frequently updated internal document set, with citations
- C. You need to reduce inference latency
- D. You need the model to learn a new language from scratch

Answer: B RAG wins when you need answers grounded in **frequently updated** documents with **citations** — you can update the document store without retraining anything. Fine-tuning is better for fixed output style/format or teaching genuinely new behavior/language.

Q14.

When an LLM performs **function/tool calling**, what actually happens?

- A. The LLM executes the function internally and returns its result
- B. The LLM returns a structured call (function name + arguments); your application executes it and sends the result back to the model
- C. The function is executed by the vector database
- D. The LLM rewrites the user's prompt into SQL and runs it directly

Answer: B The LLM does **not** execute anything itself — it returns a structured request (function name + arguments), your application code actually runs it, and the result is sent back to the model to continue reasoning. This is the single most important agentic AI concept — know it cold.

Q15.

In n8n, you want an external system to trigger a workflow the moment an event occurs, then have the workflow call a third-party API and return a response to the caller. Which combination is correct?

- A. Schedule Trigger → Code node → Wait node
- B. Webhook node (trigger) → HTTP Request node → Respond to Webhook node
- C. Manual Trigger → Webhook node → Set node
- D. Cron node → Respond to Webhook node → HTTP Request node

Answer: B Webhook = receives the external trigger; HTTP Request = calls the third-party API; “Respond to Webhook” = sends a response back to whoever called it. This is the standard n8n pattern for building an API-like automation.

SECTION B — Theory & Coding (5 × 15 = 75 marks)

Q16. End-to-End Machine Learning Pipeline (15 marks)

You are given `customers.csv` for a telecom churn problem. It contains: - Numeric columns: `tenure_months`, `monthly_charges`, `total_charges` (has missing values) - Categorical columns: `contract_type`, `payment_method`, `internet_service` - Target: `churn` (Yes/No) — only **18%** of customers churned

(a) Write Python code using pandas and scikit-learn that loads the data, splits it into train and test sets correctly, and builds a **single Pipeline** that handles missing-value imputation, scaling of numeric features, and encoding of categorical features (use `ColumnTransformer`), ending in a classifier of your choice. (6 marks)

Answer:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv("customers.csv")
X = df.drop(columns=["churn"])
y = df["churn"].map({"Yes": 1, "No": 0})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

numeric_features = ["tenure_months", "monthly_charges", "total_charges"]
categorical_features = ["contract_type", "payment_method", "internet_service"]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42))
])

pipeline.fit(X_train, y_train)

```

(b) Explain why the preprocessing must live inside the Pipeline rather than being applied to the full DataFrame before splitting. Name the specific problem this prevents and describe how it would inflate your reported score. (4 marks)

Answer: This prevents **data leakage**. If you scale/impute on the full DataFrame before splitting, statistics from the test set (its mean, its most frequent category, etc.) influence the transformation applied to training data. The model then implicitly “knows” something about the test distribution before ever training, which **inflates your reported score** — it looks better than it will perform on truly unseen data in production.

(c) Given the class imbalance, state which evaluation metric(s) you would report and justify your choice in the business context of churn. Then write code that performs hyperparameter tuning using cross-validation with that metric as the scoring function. (5 marks)

Answer: With only 18% churners, **accuracy is misleading**. Report **Recall, Precision, F1, and PR-AUC** — in a churn business context, missing an actual churner (false negative) usually costs more than a false alarm, so **Recall** (or F1 if you need balance) is usually prioritized.

```

from sklearn.model_selection import GridSearchCV

param_grid = {
    "classifier__n_estimators": [100, 200],
    "classifier__max_depth": [5, 10, None]
}

grid_search = GridSearchCV(
    pipeline, param_grid, scoring="f1", cv=5, n_jobs=-1
)
grid_search.fit(X_train, y_train)

```

Q17. Deep Learning with PyTorch/TensorFlow (15 marks)

You must build an image classifier for a 10-class dataset of 64×64 RGB images.

(a) Write the code for a CNN model class with at least two convolutional blocks (convolution → activation → pooling) followed by fully connected layers. Clearly show the input and output dimensions of each layer, and state how you arrived at the flattened feature size before the first dense layer. (6 marks)

Answer:

```

import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Input: (batch, 3, 64, 64)
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1) # -> (batch, 16, 64, 64)
        self.pool1 = nn.MaxPool2d(2) # -> (batch, 16, 32, 32)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1) # -> (batch, 32, 32, 32)
        self.pool2 = nn.MaxPool2d(2) # -> (batch, 32, 16, 16)
        self.relu = nn.ReLU()

        # Flattened size: 32 channels * 16 * 16 = 8192
        self.fc1 = nn.Linear(32 * 16 * 16, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool1(self.relu(self.conv1(x)))
        x = self.pool2(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1) # flatten
        x = self.relu(self.fc1(x))
        return self.fc2(x) # raw logits - no softmax here

```

With padding=1, kernel=3, convolution preserves spatial size; each MaxPool2d(2) halves it. Starting at 64×64 → after two pools → 16×16, with 32 channels → flatten to 32×16×16 = 8192.

(b) Write a complete training loop for one epoch, including the forward pass, loss computation, backward pass, optimizer step, and a separate validation pass. Explain the purpose of optimizer.zero_grad(), loss.backward(), optimizer.step(), and why the validation pass is run under torch.no_grad() /in evaluation mode. (6 marks)

Answer:

```
def train_one_epoch(model, train_loader, val_loader, optimizer, criterion, device):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()          # clear old gradients from last step
        outputs = model(images)         # forward pass
        loss = criterion(outputs, labels)
        loss.backward()                 # backward pass – computes gradients
        optimizer.step()                # updates weights using those gradients

    model.eval()
    val_loss = 0.0
    with torch.no_grad():               # no gradient tracking needed for validation
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            val_loss += criterion(outputs, labels).item()
    return val_loss / len(val_loader)
```

- `zero_grad()` : PyTorch accumulates gradients by default, so you must clear them each step or they'd add up across batches.
- `loss.backward()` : computes gradients of the loss w.r.t. every parameter (backpropagation).
- `optimizer.step()` : actually updates the weights using those gradients.
- `torch.no_grad()` + `.eval()` : disables gradient tracking (saves memory/speed) and turns off training-only layers like Dropout/BatchNorm updates, since we're not learning during validation.

(c) Your training loss keeps falling while validation loss starts rising after epoch 8. Name this phenomenon, and describe **three** distinct techniques you would apply to address it, explaining what each one actually does. (3 marks)

Answer: This is **overfitting**. Three fixes: 1. **Dropout** — randomly zeroes some activations during training, forcing the network not to rely on any single path, improving generalization. 2. **Data augmentation** — random crops/flips/rotations effectively grow the training set, so the model sees more variation and memorizes less. 3. **Early stopping** — stop training once validation loss starts rising, keeping the model from continuing to fit noise in the training set.

Q18. Generative AI — Building a RAG System (15 marks)

Your company has 5,000 internal PDF policy documents. Employees must be able to ask questions in natural language and receive answers grounded in those documents, with sources cited.

(a) Describe the full RAG architecture as two phases — **ingestion** and **query time** — listing every step in order. Explain the role of chunking and chunk overlap, and why embeddings are stored in a vector database rather than a relational table. (5 marks)

Answer: Ingestion: Load PDFs → extract text → **chunk** the text into smaller passages (with **overlap** between chunks so context isn't cut off mid-sentence/idea at chunk boundaries) → generate an embedding vector for each chunk → store (embedding + original text + metadata like source file/page) in a **vector database**.

Query time: User asks a question → embed the question with the same embedding model → **retrieve** the top-k most similar chunks from the vector DB (via cosine similarity) → construct a prompt containing the retrieved chunks as context → send to the LLM → return answer with citations back to the source chunks.

Embeddings are stored in a vector database (not a relational table) because similarity search requires fast **approximate nearest neighbor** lookups across high-dimensional vectors — something relational databases aren't built to do efficiently at scale.

(b) Write Python code for a minimal RAG implementation: chunk the text, generate embeddings, store them in a vector store (FAISS, Chroma or Qdrant), retrieve the top-k relevant chunks for a user query, and construct the final prompt sent to the LLM. Include the system prompt you would use to force the model to answer only from the retrieved context. (6 marks)

Answer:

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

embedder = SentenceTransformer("all-MiniLM-L6-v2")

def chunk_text(text, chunk_size=500, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size - overlap
    return chunks

# --- Ingestion ---
chunks = chunk_text(document_text)
embeddings = embedder.encode(chunks)
index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(np.array(embeddings))

# --- Query time ---
def retrieve(query, k=3):
    query_vec = embedder.encode([query])
    distances, indices = index.search(np.array(query_vec), k)
    return [chunks[i] for i in indices[0]]

def build_prompt(query, retrieved_chunks):
    context = "\n\n".join(retrieved_chunks)
    system_prompt = (
        "You are a helpful assistant. Answer ONLY using the context provided below. "
        "If the answer is not in the context, say you don't know. Cite which excerpt you used."
    )
    return f"{system_prompt}\n\nContext:\n{context}\n\nQuestion: {query}"
```

(c) The system sometimes returns confident but incorrect answers. Identify **three** likely causes rooted in the retrieval stage (not the LLM itself) and give a concrete fix for each. Also state one situation where RAG would be the wrong choice and fine-tuning would be better. (4 marks)

Answer: 1. **Chunking too large/small or losing context** → tune chunk size and overlap so each chunk is self-contained. 2. **Irrelevant chunks retrieved** (poor embedding match, e.g. ambiguous wording) → improve retrieval with hybrid search (keyword + semantic) or re-ranking of top-k results. 3. **Outdated or duplicate documents in the vector store** → add document versioning/deduplication and refresh the index when source docs change.

RAG is the *wrong* choice when the need is a **fixed output style/format learned from many examples**, or teaching genuinely new reasoning/behavior — that's better solved with fine-tuning.

Q19. Model Deployment & AI/ML Engineering (15 marks)

You have a trained scikit-learn model saved as `model.pkl` that must be served to a frontend team.

(a) Write a FastAPI application exposing a `POST /predict` endpoint and a `GET /health` endpoint. Use a Pydantic model to validate the incoming JSON request body, load the model once at startup (not per request), and return the prediction along with a confidence score as JSON. Include appropriate error handling. (6 marks)

Answer:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib

app = FastAPI()
model = None

class PredictRequest(BaseModel):
    feature1: float
    feature2: float
    feature3: float

@app.on_event("startup")
def load_model():
    global model
    model = joblib.load("model.pkl")  # loaded once, not per request

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(request: PredictRequest):
    try:
        features = [[request.feature1, request.feature2, request.feature3]]
        prediction = model.predict(features)[0]
        confidence = max(model.predict_proba(features)[0])
        return {"prediction": prediction, "confidence": confidence}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

(b) Write a Dockerfile to containerise this application. Explain your choice of base image, why `requirements.txt` is copied and installed **before** the application code, and how you would run the container mapping port 8000. (4 marks)

Answer:

```

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

`python:3.11-slim` keeps the image small. `requirements.txt` is copied and installed **before** the rest of the code because Docker caches layers — if only your app code changes (not dependencies), Docker reuses the cached install layer instead of reinstalling everything, speeding up rebuilds. Run with:

```

docker build -t my-model-api .
docker run -p 8000:8000 my-model-api

```

(c) Six months later you have four model versions in production usage. Explain how you would handle model versioning and experiment tracking (you may reference MLflow), how the API would serve a specific version, and what you would monitor in production to detect that a deployed model has degraded. Define data drift in your answer. (5 marks)

Answer: Use **MLflow** to log each training run's parameters, metrics, and the resulting model artifact, tagging each with a version number. The API can accept a `model_version` parameter/route (e.g. `/predict/v3`) and load the corresponding registered model from the MLflow model registry.

In production, monitor: prediction latency, error rates, and **data drift** — a change in the statistical distribution of incoming feature data compared to training data (e.g., customer behavior shifts over time),

which silently degrades model accuracy even though the model itself hasn't changed. Track drift with statistical tests (e.g., KS test) or drift-monitoring tools, alongside monitoring live accuracy/precision if ground-truth labels become available later.

Q20. Automation & Agentic AI with n8n (15 marks)

Build an automated support-ticket assistant. When a customer submits a ticket through a web form, the system must classify its urgency, look up the customer's order status from an internal REST API, draft a reply using an LLM, save the record to a database, and send the reply by email. Tickets classified as **critical** must instead be posted to a Slack channel for a human to handle.

(a) Design the n8n workflow. List the nodes in execution order, state the type of each node (Webhook, HTTP Request, AI Agent, IF/Switch, database, email, Respond to Webhook, etc.), and explain what data passes between them. A clear diagram or an ordered list is acceptable. (6 marks)

Answer: 1. **Webhook node** (trigger) — receives the ticket submission from the web form (customer message, ID, etc.) 2. **AI Agent node** — classifies urgency (e.g., low/medium/high/critical) from the ticket text 3. **IF/Switch node** — branches based on urgency: - If **critical** → **Slack node** (posts to a channel for a human) - Else → continue the automated path 4. **HTTP Request node** — calls the internal order-status API using the customer ID 5. **AI Agent node** — drafts a reply using the ticket text + order status as context 6. **Database node** (e.g., Postgres/MySQL node) — saves the ticket, classification, and drafted reply 7. **Email node** — sends the drafted reply to the customer 8. **Respond to Webhook node** — sends a confirmation response back to the web form

Data passed: the webhook payload flows forward and is progressively enriched — ticket text → + urgency label → + order status → + drafted reply — until it's logged and sent.

(b) The AI Agent node must be able to call the order-status API itself as a tool. Write the JSON schema for that tool definition (name, description, parameters) and explain the agent's reasoning loop — how the model decides to call the tool, what it receives back, and how it decides it is finished. (5 marks)

Answer:

```
{
  "name": "get_order_status",
  "description": "Retrieves the current status of a customer's order given their customer ID.",
  "parameters": {
    "type": "object",
    "properties": {
      "customer_id": {
        "type": "string",
        "description": "The unique ID of the customer whose order status is needed."
      }
    }
  },
  "required": ["customer_id"]
}
```

Reasoning loop: the AI Agent reads the ticket, decides it needs order info to respond accurately, and **returns a structured tool call** (function name + `customer_id` argument) instead of a final answer. n8n executes the actual HTTP Request node with that argument, gets the real API response, and feeds the result back into the agent's context. The agent then reasons again — if it now has enough information to fully answer, it produces the final draft reply; if not, it can call another tool. It “decides it's finished” once its output no longer includes a tool call, just a final text response.

(c) The workflow will run in production. Explain how you would handle: (i) the external API being temporarily down, (ii) duplicate webhook deliveries for the same ticket, and (iii) securing the webhook endpoint and API credentials. (4 marks)

Answer: - **External API down** — add retry logic with exponential backoff in the HTTP Request node (n8n supports retry-on-fail), and a fallback path (e.g., queue the ticket or notify support) if retries are exhausted. - **Duplicate webhook deliveries** — use an **idempotency key** (e.g., ticket ID) — check the database for that ID before processing again, so the same ticket isn't answered/emailed twice. - **Securing webhook +**

credentials — use a secret token/HMAC signature validation on the webhook endpoint, store API keys/credentials in n8n's built-in credential manager (never hard-coded in nodes), and restrict the webhook to HTTPS only.

Last-minute priority if time is short

If you only have 30 minutes: focus on **Q14, Q15, Q20** (agentic AI/n8n — this is clearly the role's core focus) and **Q4, Q6, Q13** (leakage, imbalance, RAG vs fine-tuning — these come up constantly in ML interviews). Being able to *talk through* Q20 clearly, even without perfect code, will matter most for this specific role.